January 2026 CSE 406

Lab Assignment: Side Channel Attack

Total Marks: 100 Points | **Last Update Date:** August 2, 2026

1. Overview & Learning Objectives

Side-channel attacks exploit physical or operational implementation flaws—such as power consumption, electromagnetic radiation, or execution timing—rather than mathematical weaknesses in modern cryptographic primitives. In this assignment, you will simulate a remote timing attack against a target authentication service to recover a secret PIN.

In many authentication systems, the supplied password or PIN is compared with the stored secret one character at a time. A common implementation checks the first character, then the second, and continues until a mismatch is found or the entire string has been verified. As a result, an incorrect input that differs in the very first character is rejected almost immediately, whereas an input that shares a longer correct prefix requires more comparisons before the mismatch is detected. Although the time difference for a single comparison is extremely small, it can often be measured by sending many authentication requests and analyzing the average response time. By identifying which candidate character consistently results in a slightly longer execution time, an attacker can infer the correct character of the secret. Repeating this process one position at a time allows the entire PIN or password to be recovered without directly breaking the underlying cryptographic algorithm. You will simulate a similar scenario. Note that some artificial delays are introduced in the server to make the assignment easier.

Learning Objectives:

- Understand the security risks associated with non-constant-time string comparisons in authentication routines.
- Develop automated scripts to collect, filter, and statistically analyze microsecond-level timing differences over HTTP.
- Apprehend how attacker-controlled inputs interact with short-circuiting logical loops.

2. Environment Setup & Provided Infrastructure

You are provided with a black-box compiled binary target server. The server runs locally and exposes an HTTP REST API endpoint for PIN verification.

Target Server Executable:

Download the compiled server binary matching your operating system from moodle:

- `server_windows.exe (Windows)`
- `server_mac (macOS - ensure executable permissions via `chmod +x`)`
- `server_linux (Linux - ensure executable permissions via `chmod +x`)`

When launched, the server listens on http://127.0.0.1:5000/verify. Do NOT attempt to reverse-engineer or decompile the provided binary; all required information must be extracted via HTTP side-channel timing measurements.

Dynamic Secret Generation:

The server generates a unique, deterministic 4-digit PIN for each student based on your Student ID (last 3 digit).

3. Task Specifications & Requirements

Task 1: Average Measurement

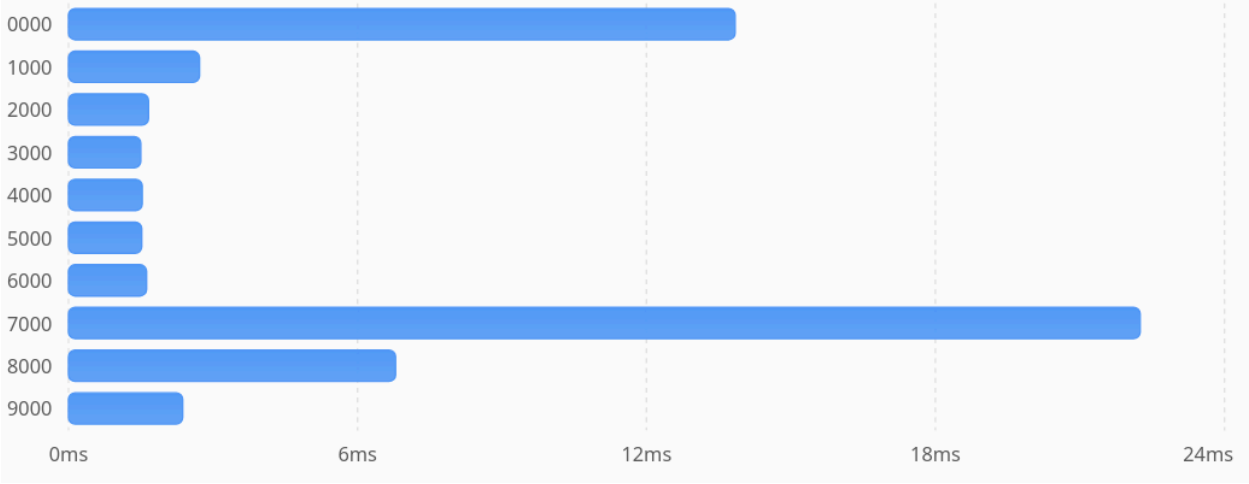
Define a method to calculate average elapsed time across a specified number of samples.

Task 2: Automated Timing Exploit Development

- Iterate through candidate digits ('0'–'9') for the current position.
- Collect multiple response-time samples (e.g., 5–10 iterations) using `time.perf_counter()` to minimize operating system context-switching noise.
- Compute average latency for each candidate digit.
- Identify the timing spike representing a correct prefix match and append the discovered digit to your candidate string.
- Automatically terminate upon receiving an HTTP 200 OK status code.

Task 3: Statistical Analysis & Visualization (20 Marks)

Collect timing data for all 10 digit attempts ('0'–'9') at each position of your PIN. Generate a bar chart comparing the average response latency across digits for each position. For example, following chart shows the time elapsed for the first position:



4. Grading Rubric

Component	Criteria	Marks
Exploit Script Functionality	Script successfully recovers the student's unique 4-digit PIN fully automated via timing measurements.	**40 Marks**
Noise Handling & Sampling	Implementation of averaging to reliably handle system jitter and prevent false positives.	**20 Marks**
Timing Visualization	Clear, correctly labeled bar plot demonstrating the timing spike for each PIN digit.	**30 Marks**
Submission	Proper submission.	**10 Marks**

5. Submission Guidelines

Submit your exploit script as <student_id>.py (e.g. 2105208.py)

6. Deadline

14 August 2026, 11:59 pm